

Day 3 - Facilitation Guide

(HTML Table and Forms)

INDEX

- I. Recap of the previous session**
- II. Div tag**
- III. HTML Tables**
- IV. Different HTML Table Tags**
- V. Forms**
- VI. Different Form Widgets**
- VII. Document type declaration**
- VIII. How to add a Form in a Table for better display**

(2 hrs) ILT

I. Recap of the previous session

In the last session we taught you how to create an HTML page and view it in the browser. We also explained what HTML Tags and elements are. We now have a good understanding on how to format our page with different types of text like bold text, italic text, subscript , superscript etc.

We also learned how to create ordered and unordered lists in HTML. An unordered list is used to create a list of items where the order of the items does not matter. The items in an unordered list are displayed with bullets (or other markers depending on the CSS styles applied) by default.

An ordered list is used to create a list of items where the order of the items is important. The items in an ordered list are displayed with numbers (or letters, or Roman numerals, depending on the type attribute or CSS styles applied) by default. Ordered lists are often used for steps in a process, rankings, or any other situation where the order matters.

In addition to that we also got an idea about the best practices used in UI design.

In this session we will learn to use the div tag as well as HTML Table and Forms.

II. Div tag

The <div> tag is an empty container that is used to define a division or a section. It does not affect the content or layout and is used to group HTML elements to be styled with CSS or manipulated with scripts.

The <div> tag in HTML stands for "division" and is a container element that is used to group other HTML elements together.

It is a block-level element, which means it typically starts on a new line and stretches the full width of its container.

Here's a simple example of how the <div> tag can be used:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Learning HTML Page</title>
  <style>
    body {
      font-family: 'Arial', sans-serif;
      margin: 0;
      padding: 0;
      background-color: #f4f4f4; /* Light Gray Background */
      color: #333;
    }
    header {
      background-color: #3498db; /* Blue Header */
      color: #fff;
      padding: 20px;
      text-align: center;
      font-size: 26px;
    }

    section {
      padding: 20px;
      margin: 20px;
      background-color: #fff; /* White Background for Content */
      border-radius: 5px;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    }

    footer {
      background-color: #3498db; /* Blue Footer */
```

```
        color: #fff;
        text-align: center;
        padding: 10px;
    }

    .container {
        max-width: 800px;
        margin: 0 auto;
    }

    h1, h2 {
        color: #3498db; /* White Heading */
    }
</style>
</head>

<body>

    <div id="header" class="container">

        <header>
        <p>Learning HTML</p>
        <p>Welcome to the world of HTML!</p>
        </header>

    </div>

    <div id="main-content" class="container">
        <section>
            <h2>Introduction</h2>
            <p>HTML (Hypertext Markup Language) is the standard markup language
for creating Web pages. It is used to create content on the web, it provides a
basic framework for the presentation of information.</p>

            <h2>Key Concepts</h2>
            <ul>
                <li>HTML Elements</li>
                <li>Tags and Attributes</li>
                <li>Document Structure</li>
                <li>Links and Images</li>
                <li>List and Table Elements</li>
            </ul>
        </section>
    </div>
```

```
<div id="footer" class="container">
  <footer>
    &copy; 2023 Learning HTML. All rights reserved.
  </footer>
</div>

</body>
</html>
```

In this example:

- The outer `<div>` with the class "container" is used to group and style content. It acts as a container for the header and the main-content `<div>`.

If we save the HTML code in a file and open in a Web browser then it will look like this:



There are many cases where `<DIV>` tag can be used effectively.

Structuring Content: <Div> is used to group and structure content within an HTML document. It allows developers to organize elements into logical sections, making the HTML code more readable and maintainable.

Applying Styles and Layouts: Developers often use <div> elements as containers to apply styles and define layouts using CSS. By assigning class or ID attributes to <div> elements, designers can target specific sections of the page for styling.

Creating Grid Systems: <Div> elements are a fundamental building block for creating grid-based layouts. Grid systems help in achieving responsive designs and organizing content into columns and rows.

JavaScript and DOM Manipulation: Developers use <div> elements as containers for JavaScript functionalities. The DOM (Document Object Model) can be manipulated using JavaScript to dynamically update or modify the content within <div> elements.

Separating Sections in HTML Document: The <div> tag helps in separating different sections of an HTML document, such as header, main content, sidebar, and footer. This separation improves code organization and makes it easier to understand the structure of the page.

Creating Modal Windows and Overlays: Modal windows and overlays, commonly used for pop-ups or lightboxes, are often implemented using <div> elements. These elements act as containers for the modal content and can be easily styled and positioned.

Grouping and Styling Form Elements: The <div> tag is also used to group and style form elements, providing a way to apply consistent styling to related input fields, labels, and buttons.

The <div> tag is versatile and is often used to create layout structures by grouping and organizing content. It provides a way to apply styles or scripts to a specific section of a Webpage, making it easier to manage and manipulate the presentation of content. It doesn't have any specific semantic meaning on its own, but it serves as a powerful tool for structuring HTML documents.

III. HTML Tables

HTML tables are used to organize and display data in rows and columns. They provide a way to structure content in a tabular format, making it easy to present information in a structured manner. HTML tables consist of several elements that define the structure and content of the table. Here's an overview of HTML tables:

HTML Tables consist of rows and columns. And all rows and columns can contain table data.

Name	Age	City
John Doe	25	New York
Jane Smith	30	London
Bob Johns	22	Paris

← Table Header

Table Row

Table Data

Basic Table Structure:

The basic structure of an HTML table involves the use of the following elements:

<table>: The container element for the entire table.

<tr>: Defines a table row. It contains one or more <td> or <th> elements.

<td>: Represents a table cell (data cell) within a row.

<th>: Represents a table header cell. It is used to label columns or rows and is typically bold and centered.

Here is a view of a table in an HTML page.

The screenshot shows a web browser window with the title 'HTML Table with Border'. The address bar shows '127.0.0.1:5500...'. The main content area displays a table with the following data:

Name	Age	City
John Doe	25	New York
Jane Smith	30	London
Bob Johnson	22	Paris

IV. Different HTML Table Tags

There are various types of HTML table tags and attributes to define a table.

<table>: Defines the start of the table.

<tr>: Defines a table row. It contains one or more cells.

<th>: Defines a table header cell. Typically used in the first row to label columns or rows.

<td>: Defines a regular table data cell. Contains the actual data.

<caption>: It defines the table caption.

<colgroup>: It specifies a group of one or more columns in a table for formatting.

<col>: It is used with <colgroup> element to specify column properties for each column.

<tbody>: It is used to group the body content in a table.

<thead>: It is used to group the header content in a table.

<tfoot>: It is used to group the footer content in a table.

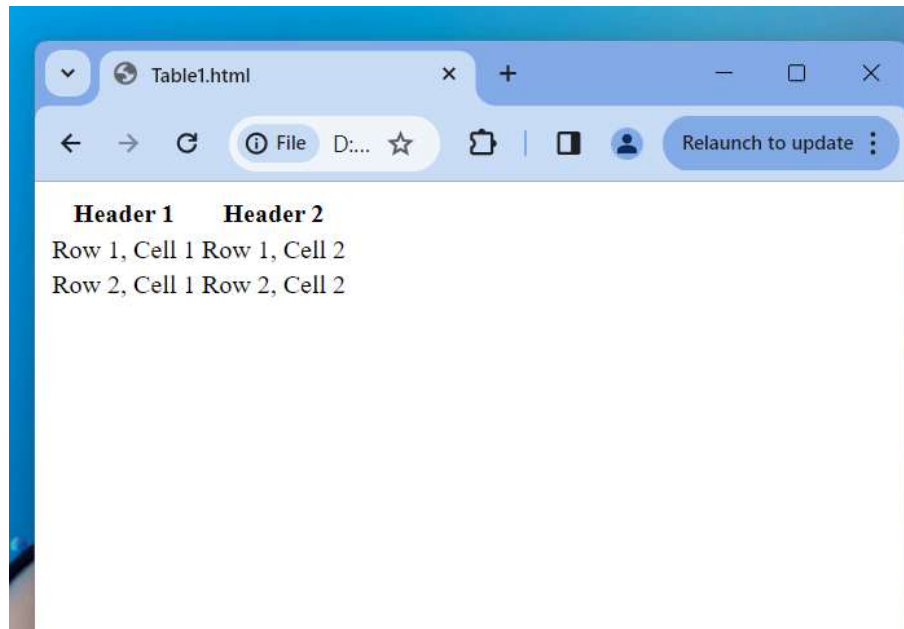
HTML tables are versatile and can be customized further using CSS for styling. While tables were traditionally used for layout purposes, modern Web design often prefers other layout techniques like flexbox or grid for non-tabular content. Tables are still widely used for displaying tabular data.

Here's a basic example of an HTML table:

```
<html>
  <body>
    <table>
      <tr>
        <th>Header 1</th>
        <th>Header 2</th>
      </tr>
      <tr>
        <td>Row 1, Cell 1</td>
        <td>Row 1, Cell 2</td>
      </tr>
      <tr>
        <td>Row 2, Cell 1</td>
        <td>Row 2, Cell 2</td>
      </tr>
    </table>
  </body>
</html>
```

In this example, we have a table with two rows and two columns. The first row contains header cells (<th>), and the subsequent rows contain regular data cells (<td>).

Opening in browser will look like this:



Attributes:

colspan and rowspan: These attributes allow a cell to span multiple columns (colspan) or rows (rowspan).

border: Specifies the width of the border around the table and its cells. It can take values like 1 for a thin border.

width and height: Define the width and height of the table or its cells.

align: Specifies the horizontal alignment of the table or its cells. Common values are "left", "center", and "right".

bgcolor: Sets the background color of the table or its cells.

cellpadding and cellspacing: These attributes define the space between the cell content and the cell border (cellpadding) and the space between cells (cellspacing).

Example of table with Attributes:

```
<html>
  <body>
    <table border="1" width="50%" cellspacing="0" cellpadding="10">
      <tr>
        <th colspan="2">Header Cells Spanning 2 Columns</th>
      </tr>
      <tr>
        <td>Row 1, Cell 1</td>
        <td>Row 1, Cell 2</td>
      </tr>
      <tr>
        <td bgcolor="#FF0000">Row 2, Cell 1 with Red Background</td>
        <td rowspan="2">Row 2, Cell 2<br>Spanning 2 Rows</td>
```



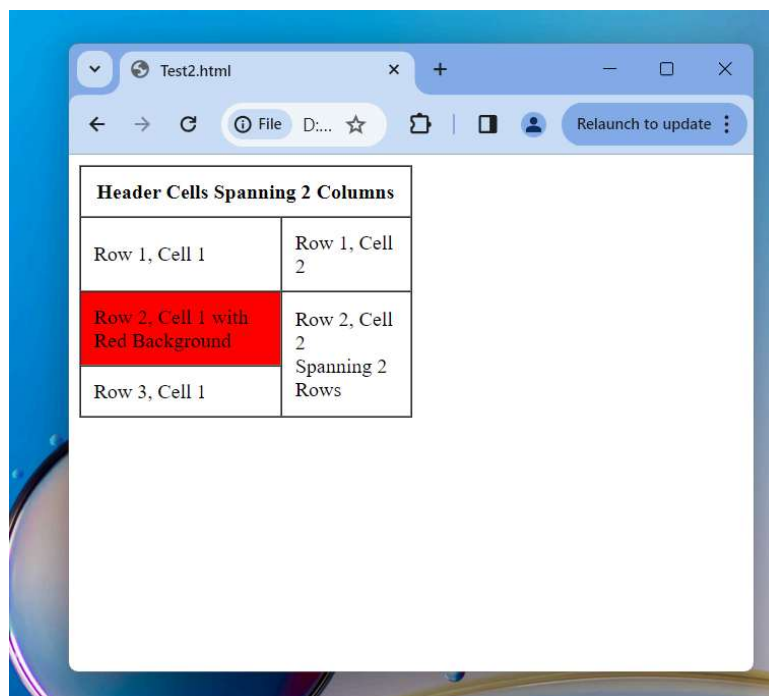
```

        </tr>
        <tr>
            <td>Row 3, Cell 1</td>
        </tr>
    </table>
</body>
</html>

```

In this example, the table has various attributes set to customize its appearance.

Opening in browser will look like this:



We can also apply style tags to add better look and feel to HTML. Here is a simple HTML table with style applied.

```

<!DOCTYPE html>
<html lang="en">

```

```

<head>

```

```

    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Styled Table</title>
    <style>

```

```
body {
    font-family: 'Arial', sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4; /* Light Gray Background */
    color: #333;
}

header {
    background-color: #3498db; /* Blue Header */
    color: #fff;
    padding: 20px;
    text-align: center;
}

section {
    padding: 20px;
    margin: 20px;
    background-color: #fff; /* White Background for Content */
    border-radius: 5px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}

th, td {
    border: 1px solid #ddd; /* Light Gray Border */
    padding: 12px;
    text-align: left;
}

th {
    background-color: #3498db; /* Blue Header Color */
    color: #fff;
    font-weight: bold;
}

tr:nth-child(even) {
    background-color: #f9f9f9; /* Light Gray Background for Even Rows */
}

tr:hover {
```

```

        background-color: #e0e0e0; /* Darker Gray Background on Hover */
    }

    footer {
        background-color: #3498db; /* Blue Footer */
        color: #fff;
        text-align: center;
        padding: 10px;
    }
</style>
</head>

<body>

    <header>
        <h1>Styled Table</h1>
        <p>Enjoy the beauty of a styled HTML table!</p>
    </header>

    <section>
        <table>
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Age</th>
                    <th>Occupation</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>John Doe</td>
                    <td>28</td>
                    <td>Software Developer</td>
                </tr>
                <tr>
                    <td>Jane Smith</td>
                    <td>35</td>
                    <td>Graphic Designer</td>
                </tr>
                <tr>
                    <td>Robert Johnson</td>
                    <td>42</td>
                    <td>Teacher</td>
                </tr>
            </tbody>
        </table>
    </section>

```

</section>

<footer>

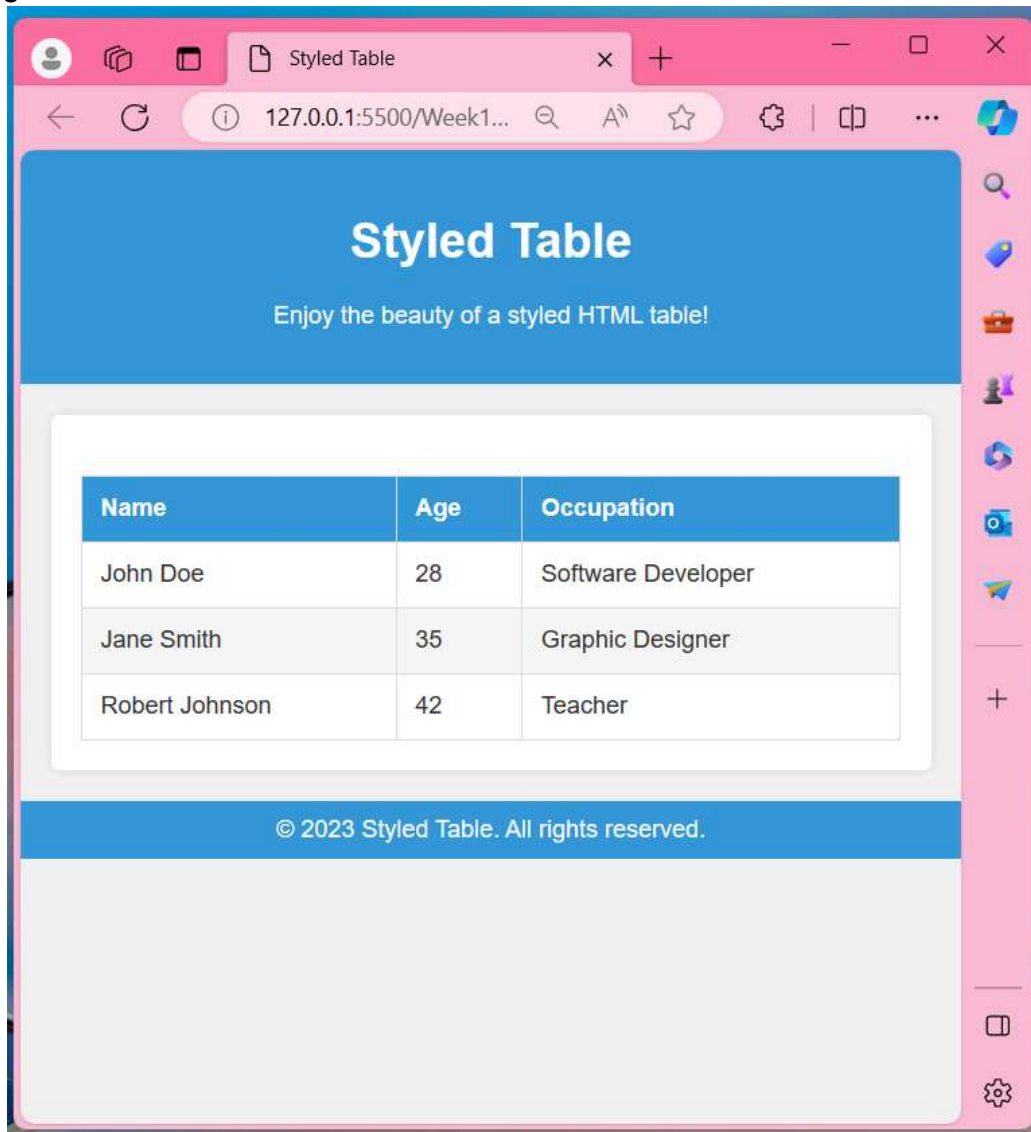
© 2023 Styled Table. All rights reserved.

</footer>

</body>

</html>

Opening in browser will look like this:



Remember that tables should be used for displaying tabular data.

For layout purposes, CSS and other HTML elements like `<div>` are more appropriate. Proper use of tables ensures accessibility and better **SEO** (Search Engine Optimization) for your Web pages.

Note: SEO stands for “search engine optimization.” In simple terms, SEO means the process of improving your Website to increase its visibility in Google, Microsoft Bing, and other search engines whenever people search for it.

V. Forms

In HTML, a form is a user interface element used to collect user input. Forms enable users to input various types of data, such as text, numbers, passwords, options, and more, which can be submitted to a server for further processing. Forms are essential for interactive Websites, allowing users to provide information, make selections, and perform actions like submitting orders, filling out surveys, and logging in to accounts.

A form consists of one or more input elements, along with other elements like buttons, checkboxes, radio buttons, and text areas, which allow users to enter data or make selections.

When the user submits the form, the data is sent to a server-side script or another Web page whose path is specified in the form's **action** attribute for processing.

The server-side script can then handle the submitted data, perform necessary actions, and send a response back to the user.

Here's a basic structure of an HTML form:

```
<form action="/submit" method="POST">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required><br>

    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required><br>

    <input type="submit" value="Submit">
</form>
```

VI. Different Form Widgets

In HTML, form widgets are elements that allow users to interact with the form by providing input, making selections, and performing actions.

There are several types of form widgets available, each serving a specific purpose. Here's an overview of the different form widgets in HTML:

Text Input (<input type="text">):

Allows users to enter single-line text. Example: username, email, etc.

```
<input type="text" name="username">
```

Password Input (<input type="password">):

Similar to text input but hides the characters typed by the user. Example: password fields.

```
<input type="password" name="password">
```

Checkbox (<input type="checkbox">):

Allows users to select one or more options from a list of choices.

```
<input type="checkbox" name="interest" value="sports"> Sports  
<input type="checkbox" name="interest" value="music"> Music
```

Radio Button (<input type="radio">):

Allows users to select a single option from a list of choices.

```
<input type="radio" name="gender" value="male"> Male  
<input type="radio" name="gender" value="female"> Female
```

Dropdown Menu (<select> and <option>):

Allows users to select one option from a list of choices.

```
<select name="country">  
  <option value="usa">United States</option>  
  <option value="canada">Canada</option>  
  <option value="uk">United Kingdom</option>  
</select>
```

Textarea (<textarea>):

Allows users to enter multi-line text, suitable for longer responses or comments.

```
<textarea name="comments" rows="4" cols="50"></textarea>
```

File Input (<input type="file">):

Allows users to select and upload files from their devices.

```
<input type="file" name="file">
```

Submit Button (<input type="submit">):

Submits the form data to the server-side script specified in the form's action attribute.

```
<input type="submit" value="Submit">
```

Reset Button (<input type="reset">):

Resets all form fields to their default values.

```
<input type="reset" value="Reset">
```

Button (<button>):

Can be used for custom actions or interactivity within the form.

```
<button type="button">Click Me</button>
```

In HTML forms, both the `<input type="submit">` and `<button>` elements can be used to create buttons that allow users to submit the form. However, there are some differences in their behavior and use cases. Here are the key distinctions between `<input type="submit">` and `<button>`:

<input type="submit">: This is a specific input element designed for form submission. It's explicitly created for submitting forms, and its primary purpose is to trigger the form's submission when clicked.

```
<form action="/submit" method="post">
  <input type="submit" value="Submit Form">
</form>
```

If you need to interact with the button using JavaScript, you might need to select it using its type attribute.

```
<script>
  document.querySelector('input[type="submit"]').addEventListener('click',
function() {
  alert('Form submitted!');
});
</script>
```

<button>: This is a more generic button element that can be used for various purposes, including form submission. It can contain text, images, or other HTML elements, providing more flexibility in terms of content.

```
<form action="/submit" method="post">
  <button type="submit">Submit Form</button>
</form>
```

Interaction with a button using JavaScript is straightforward, and you can reference it by its tag name.

```
<script>
  document.querySelector('button').addEventListener('click', function() {
```

```
        alert('Form submitted!');
    });
</script>
```

In summary, both `<input type="submit">` and `<button>` can be used to create a form submission button, but the choice depends on factors such as flexibility in content, accessibility requirements, and JavaScript interactions.

The `<button>` element is often preferred for its versatility and improved accessibility features.

These form widgets can be used individually or in combination within a `<form>` element to create interactive and user-friendly Web forms.

Note: Properly designing forms with appropriate widgets and labels enhances user experience and ensures accurate data collection.

Here is an HTML page that contains various form widgets:

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Form Widgets Example</title>
  </head>

  <body>
    <h2>Form Widgets Example</h2>

    <form action="/submit" method="POST">
      <label for="text-input">Text Input:</label>
      <input type="text" id="text-input" name="text_input"><br>

      <label for="password-input">Password Input:</label>
      <input type="password" id="password-input" name="password_input"><br>

      <label>Checkbox:</label>
      <input type="checkbox" name="interest" value="sports"> Sports
      <input type="checkbox" name="interest" value="music"> Music<br>

      <label>Radio Buttons:</label>
      <input type="radio" name="gender" value="male"> Male
      <input type="radio" name="gender" value="female"> Female<br>

      <label for="country">Dropdown Menu:</label>
```



```
<select id="country" name="country">
  <option value="usa">United States</option>
  <option value="canada">Canada</option>
  <option value="uk">United Kingdom</option>
</select><br>

<label for="comments">Textarea:</label>
<textarea id="comments" name="comments" rows="4"
cols="50"></textarea><br>

<label for="file-input">File Input:</label>
<input type="file" id="file-input" name="file_input"><br>

<input type="submit" value="Submit">
<input type="reset" value="Reset">

<button type="button">Custom Button</button>
</form>
</body>
</html>
```

Opening in the Web browser will look like this:

Form Widgets Example

Text Input:

Password Input:

Checkbox: ☐ Sports ☐ Music

Radio Buttons: ☐ Male ☐ Female

Dropdown Menu:

Textarea:

File Input: No file chosen

We can apply styles to make the form visually more attractive. Here is the same form with applied style.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Widgets Example</title>
  <style>
    body {
      font-family: 'Arial', sans-serif;
      background-color: #F5D6C8;
      margin: 0;
      padding: 0;
      display: flex;
      align-items: center;
      justify-content: center;
      height: 100vh;
    }
  </style>
</head>
<body>
  <div>
    <h2>Form Widgets Example</h2>
    <div>
      Text Input: <input type="text">
      Password Input: <input type="password">
      Checkbox: <input type="checkbox"> Sports <input type="checkbox"> Music
      Radio Buttons: <input type="radio"> Male <input type="radio"> Female
      Dropdown Menu: <input type="text" value="United States">
      Textarea: <input type="text">
      File Input: <input type="button" value="Choose File"> No file chosen
      <div>
        <input type="button" value="Submit">
        <input type="button" value="Reset">
        <input type="button" value="Custom Button">
      </div>
    </div>
  </div>
</body>
</html>
```

```

}

form {
    background-color: #F5D6C8;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    width: 1500px;
    text-align: left;
}

h2 {
    color: #288038;
    text-align: left;
    font-weight: bold;
}

label {
    font-weight: bold;
    display: block;
    margin: 10px 0 5px;
    color: #4B200D;
}

input[type="text"],
input[type="password"],
textarea,
select,
input[type="file"] {
    width: 300px;
    padding: 8px;
    margin: 5px 0 15px;
    box-sizing: border-box;
    border: 1px solid #ccc;
    border-radius: 4px;
}

input[type="checkbox"],
input[type="radio"] {
    margin-right: 5px;
}

input[type="submit"],
button {
    background-color: #288038;
    color: #fff;

```

```

padding: 10px 15px;
border: none;
border-radius: 4px;
cursor: pointer;
}

input[type="submit"]:hover,
button:hover {
    background-color: #EFA686;
;
}
</style>
</head>

<body>
  <form action="/submit" method="POST">
    <h2>Survey Form</h2>

    <label for="text-input">First Name</label>

    <input type="text" id="text-input" name="text_input">

    <label for="password-input">Last Name</label>

    <input type="password" id="password-input" name="password_input">

    <label>Hobbies</label>

    <input type="checkbox" name="interest" value="sports"> Sports
    <input type="checkbox" name="interest" value="music"> Music
    <input type="checkbox" name="interest" value="adventure"> Adventure

    <label>Gender</label>

    <input type="radio" name="gender" value="male"> Male
    <input type="radio" name="gender" value="female"> Female

    <label for="country">Country</label>

    <select id="country" name="country">
      <option value="usa">United States</option>
      <option value="canada">Canada</option>

```

```

        <option value="uk">United Kingdom</option>
    </select>

    <label for="comments">Comment</label>

    <textarea id="comments" name="comments" rows="4"
    cols="50"></textarea>

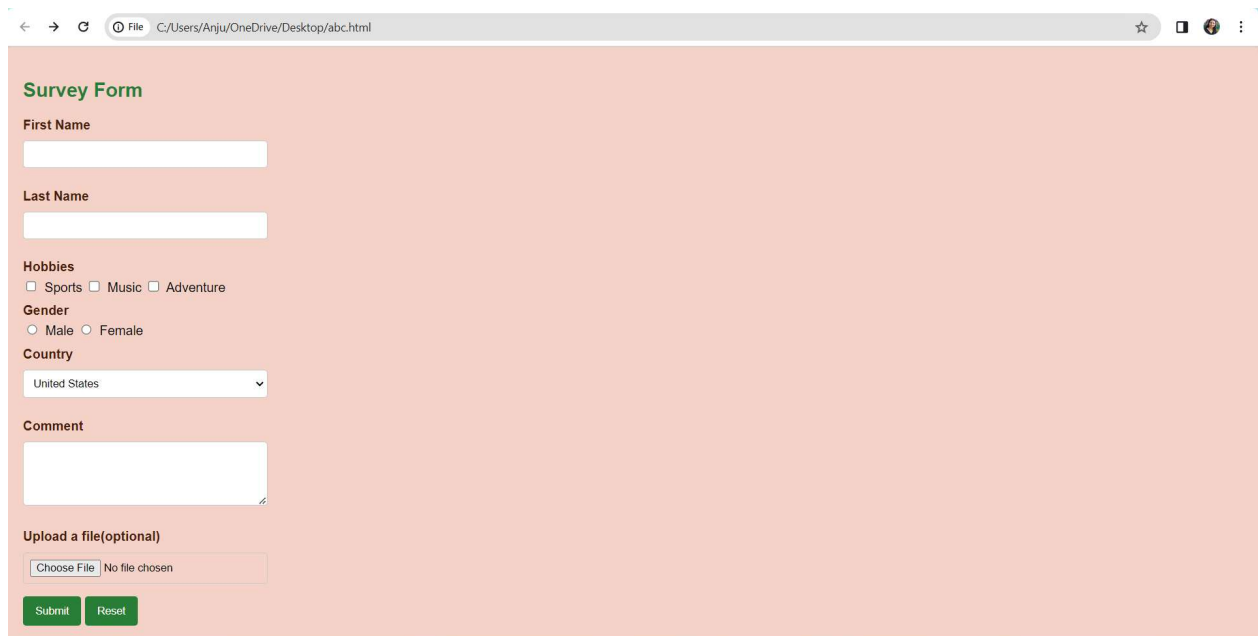
    <label for="file-input">Upload a file(optional)</label>

    <input type="file" id="file-input" name="file_input">
<br/>
    <input type="submit" value="Submit">
    <button type="button">Reset</button>
</form>
</body>

</html>

```

Opening in the Web browser will look like this:



The screenshot shows a web browser window with the address bar displaying 'C:/Users/Anju/OneDrive/Desktop/abc.html'. The page content is a 'Survey Form' with a light orange background. The form includes the following elements:

- First Name:** A text input field.
- Last Name:** A text input field.
- Hobbies:** Three checkboxes labeled 'Sports', 'Music', and 'Adventure'.
- Gender:** Two radio buttons labeled 'Male' and 'Female'.
- Country:** A dropdown menu currently showing 'United States'.
- Comment:** A text area with a small icon in the bottom right corner.
- Upload a file(optional):** A file upload control showing 'Choose File' and 'No file chosen'.
- Buttons:** Two green buttons at the bottom labeled 'Submit' and 'Reset'.

VII. Document type declaration

A Document Type Declaration (DOCTYPE) is an instruction or statement in an HTML, XML, or SGML document that defines the document type and its version. In the context

of HTML, the DOCTYPE declaration informs the Web browser about the version of HTML being used in the document, which helps the browser render the content correctly. It also defines the document type rules and syntax that the document must adhere to.

The DOCTYPE declaration is placed at the very beginning of an HTML document, before the <html> tag. In modern HTML documents (specifically HTML5), the DOCTYPE declaration for HTML is simplified and looks like this:

```
<!DOCTYPE html>
```

Here,

<!DOCTYPE>: Indicates the start of the DOCTYPE declaration.

html: Specifies the document type, which is HTML.

This declaration informs the browser that the document is written in HTML5. Unlike earlier versions of HTML, HTML5 does not require a complex DOCTYPE declaration. The above declaration is sufficient for most HTML5 documents. HTML5 will be declared in more detail later.

When you create a new HTML document, it's a good practice to include the DOCTYPE declaration as the very first line in your HTML file. It helps ensure that the browser interprets your HTML code correctly according to the specified HTML version rules.

For example, a complete HTML document with the DOCTYPE declaration might look like this:

```
<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My HTML Document</title>
  </head>

  <body>
    <h1>Hello, World!</h1>
    <p>This is a sample HTML document.</p>
  </body>

</html>
```

In this example, <!DOCTYPE html> indicates that the document is written in HTML5. Including the DOCTYPE declaration ensures that the browser renders the document following HTML5 rules and standards.

VIII. How to add a Form in a Table for better display

Adding a form within a table is a common practice when you want to create structured layouts for collecting data. By placing form elements inside table cells, you can organize input fields, labels, and buttons neatly. Here's an example of how to add a form within a table for better display:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form in Table Example</title>
</head>

<body>
  <h2>Form in Table Example</h2>

  <table border="1">
    <tr>
      <td>
        <form action="/submit" method="POST">
          <label for="username">Username:</label><br>
            <input type="text" id="username" name="username"
required><br><br>

          <label for="password">Password:</label><br>
            <input type="password" id="password" name="password"
required><br><br>

          <label for="email">Email:</label><br>
            <input type="text" id="email" name="text" required><br><br>

          <input type="submit" value="Submit">
        </form>
      </td>
      <td>
        <h3>Table Content Goes Here</h3>
        <p>This is some content inside the table cell.</p>
      </td>
    </tr>
  </table>
</body>
```

</html>

In this example:

The `<form>` element is placed inside the first table cell (`<td>`).

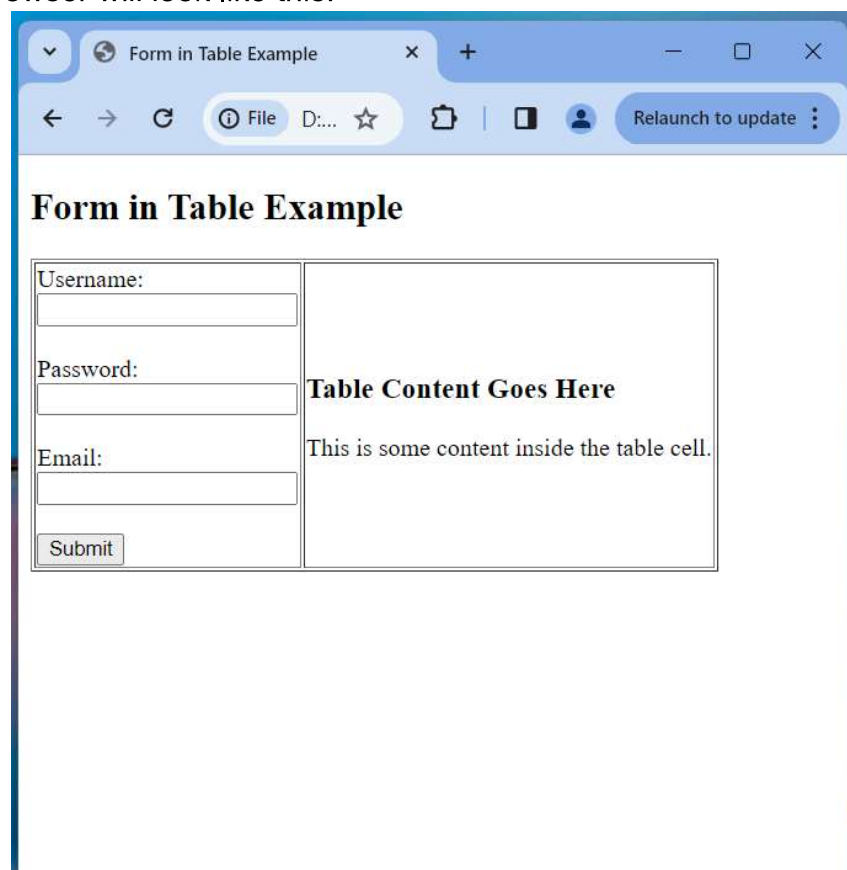
Inside the form, there are text input fields for username, password, and email, along with corresponding `<label>` elements for better accessibility.

The form also contains a submit button (`<input type="submit">`).

The table has two rows and two columns. The first column contains the form, and the second column contains some content.

By using tables in this way, you can create organized and visually appealing forms while maintaining a structured layout. However, keep in mind that tables should be used for tabular data, not for layout purposes. For complex layouts, it's generally recommended to use CSS for styling and positioning elements. Tables are best suited for displaying data in rows and columns.

Opening in browser will look like this:



We can further apply style tags to enhance the appearance of our above form within a table. Here is the HTML code.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
  <head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width,  
initial-scale=1.0">
```

```
    <title>Form in Table Example</title>
```

```
    <style>
```

```
      body {
```

```
        font-family: 'Arial', sans-serif;
```

```
        background-color: #f4f4f4;
```

```
        margin: 5px;
```

```
        padding: 5px;
```

```
        align-items: center;
```

```
        justify-content: center;
```

```
        height: 100vh;
```

```
      }
```

```
      h1 {
```

```
        color: #000000;
```

```
        text-align: center;
```

```
      }
```

```
      h2 {
```

```
        color: #3498db;
```

```
        text-align: center;
```

```
      }
```

```
      table {
```

```
        width: 500px;
```

```
        border-collapse: collapse;
```

```
        margin-top: 20px;
```

```
        align-content: center;
```

```
        align-self: center;
```

```
        margin-left:auto;
```

```
        margin-right:auto;
```

```
      }
```

```
      td {
```

```
        padding: 20px;
```

```
        text-align: center;
```

```
        vertical-align: top;
```

```
      }
```

```
form {
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  text-align: left;
  align-items: center;
}

label {
  display: block;
  margin-bottom: 5px;
  color: #000000;
}

input {
  width: 100%;
  padding: 8px;
  margin-bottom: 15px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}

input[type="submit"] {
  background-color: #F6C80A;
  color: #000000;
  padding: 10px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

input[type="submit"]:hover {
  background-color: #2980b9;
}

h3 {
  color: #3498db;
}

p {
  color: #555;
}
</style>
```

```

</head>

<body>
  <h1>Login to Amazon</h1>
  <br><br>
  <table border="1">
    <tr>
      <td>
        <form action="/submit" method="POST">
          <label for="username">Username:</label><br>
          <input type="text" id="username" name="username"
required><br><br>

          <label for="password">Password:</label><br>
          <input type="password" id="password" name="password"
required><br><br>

          <input type="submit" value="Login">

          <label for="username">Forgot Password?</label><br>
          <input type="submit" value="Create new Account">
        </form>
      </td>
      <td>
        <h1>Welcome to </h1>

        <!-- Image Link -->
        <a href="https://www.amazon.com" target="_blank">
          
        </a>
      </td>
    </tr>
  </table>
</body>

</html>

```

Opening in browser will look like this:

Login to Amazon

Username:

Password:

Login

Forgot Password?

Create new Account

Welcome to



ChatGPT Exercise:

Please visit the site - <https://chat.openai.com/> and create your account if not yet created. Next, start with the first question on the prompt:

I am learning HTML. Could you explain the advantages of using Forms in an HTML page?

I am learning HTML. Could you explain the advantages of using Forms in an HTML page?

|



Submit the query clicking the green arrow icon. Now go through the response received from the AI engine. Output will be something like the following screenshot.



Certainly! Forms in HTML are a fundamental way to collect user input. They provide several advantages, making them essential for interactive web applications:

1. **User Input Collection:** Forms allow you to collect various types of user input such as text, passwords, checkboxes, radio buttons, and more. This input is crucial for interactive websites, online surveys, registration pages, and applications that require user interaction.
2. **Data Validation:** Forms allow you to specify input requirements, such as minimum and maximum lengths, required fields, and specific data formats (like email addresses). This helps in ensuring that the data submitted by users meets certain criteria, improving data accuracy.

After the 2 hr ILT, the student has to do a 1 hour live lab. Please refer to the Session 3 Lab doc in the LMS.